UNIVERSIDADE FEDERAL DO RIO GRANDE – FURG
Centro de Ciências Computacionais – C3
Disciplina de Compiladores – Semestre 2026/2
Prof. Dr. Gerson Alberto Leiria Nunes

Trabalho Prático 1

Front-End de Compilador Pascal-S com Python PLY

Relatório técnico: decisões de projeto, gramática e análise de conflitos do parser

Integrantes: Manoela Rodrigues Sedrez e Sofia Botesini

Rio Grande, outubro de 2026

1 Introdução

Este relatório documenta o front-end de um compilador inspirado no Pascal-S, implementado em Python com a biblioteca PLY. A implementação parte dos requisitos do enunciado e acrescenta construções de linguagem para testar situações mais complexas: declarações de tipos, `boolean`, `string`, vetores e matrizes, registros, ponteiros, unidades `uses`, funções e procedimentos. O front-end reúne análise léxica sob demanda, tabela de símbolos com ambientes encadeados, análise sintática LALR(1), verificação semântica e recuperação de erros sintáticos em modo pânico. As extensões são identificadas separadamente dos requisitos mínimos para deixar explícito o escopo efetivamente implementado.

As seções seguintes descrevem a arquitetura, as decisões léxicas e semânticas, a gramática implementada, as extensões em relação ao enunciado, a análise do conflito shift/reduce observado em `parser.out`, os testes registrados e as limitações conhecidas.

2 Arquitetura

Módulo	Responsabilidade
`lex.py`	Tokens, palavras reservadas, comentários, linha/coluna, erros léxicos; expõe a fonte de tokens sob demanda (`getNextToken`).
`symtab.py`	Symbol, SymbolTable (ambientes encadeados por parent) e ScopeManager (pilha de ambientes).
`yacc.py`	Gramática LALR(1), ações semânticas (tipos), p_error com modo pânico, função run_file e linha de comando.
`run_tests.py`	Executa testes/*.pas e grava a saída real em logs/*.log.

Fluxo de dados. O parser é o único "motor": a cada vez que precisa de um lookahead ele invoca `tokenfunc=stream.getNextToken`, que delega a `lexer.getNextToken()` (apelido de `lexer.token()` do PLY). Nenhum vetor de tokens é construído: cada token é produzido, consumido e descartado. Ao produzir um token `ID`, o lexer chama `lookup_or_install` na tabela de símbolos e coloca o próprio objeto `Symbol` (uma referência) no atributo do token. O parser consulta e atualiza esse mesmo objeto nas ações semânticas, de modo que a tabela é, de fato, o repositório central de atributos.

Leitura do arquivo. O arquivo-fonte é lido de uma só vez para uma string (`read()`), porque a interface `lexer.input()` do PLY trabalha sobre uma cadeia. O que é preguiçoso é a *tokenização*: o texto só é varrido na medida em que o parser pede o próximo token. Uma consequência prática é que, se a análise é abortada, o restante do arquivo nunca é tokenizado (ver seção 6 e o teste `lex_extra.pas`).

3 Análise léxica

Tokens. O analisador reconhece os tokens básicos do enunciado e tokens adicionais necessários às extensões implementadas. Além das palavras reservadas básicas (`program`, `var`, `integer`, `real`, `begin`, `end`, `if`, `then`, `else`, `while`, `do`, `writeln`, `not`, `and`, `or`),

há palavras para tipos, registros, vetores, ponteiros, unidades, subprogramas e controle de fluxo, como `type`, `array`, `of`, `record`, `boolean`, `string`, `function`, `procedure`, `uses`, `true`, `false`, `nil`, `new`, `dispose`, `const`, `for`, `to`, `downto`, `repeat`, `until`, `div` e `mod`. Também são reconhecidos identificadores, números inteiros e reais, literais de texto, operadores e delimitadores.

Espaços em branco. Espaço, tabulação e retorno de carro (`t_ignore`: espaço, `\t` e `\r`) são descartados sem gerar token; a quebra de linha tem regra própria, que atualiza o contador de linhas.

Palavras reservadas. Não há uma expressão regular por palavra. O lexema é casado como identificador e depois filtrado em uma tabela hash (`reserved.get(valor.lower())`), o que também dá insensibilidade a maiúsculas/minúsculas (`BEGIN`, `Begin` e `begin` são o mesmo token).

Maior prefixo (longest match). Para tokens definidos por string, o PLY ordena as expressões pelo comprimento da regex, em ordem decrescente; assim `:=` vence `:` e `<=`/`<>` vencem `<`. Entre as funções vale a ordem de declaração: `NUM_REAL` é testado antes de `NUM_INT` (para que `3.14` não vire `3`, `.`, `14`) e os comentários são testados antes de `DIVIDE` (para que `//` não seja duas divisões).

Comentários. São descartados sem retornar token: `//` até o fim da linha e `{ ... }`, que pode ocupar várias linhas (o contador de linhas é incrementado pelo número de quebras dentro do comentário). Um `{` sem `}` correspondente gera erro léxico com linha e coluna, em vez de engolir silenciosamente o resto do arquivo.

Linhas e colunas. A regra de quebra de linha incrementa `lexer.lineno`. A coluna é calculada por `find_column` a partir de `lexpos` e da última quebra de linha. Todo erro léxico ou sintático informa linha, coluna e o token ou caractere ofensor.

Erros léxicos. Um caractere fora do alfabeto (por exemplo `@`, `$`, `#`) é reportado e descartado (`lexer.skip(1)`), e a análise prossegue; os erros ficam registrados em `lexer.errors` para que o resultado final seja FALHA mesmo que o parser aceite o resto.

4 Tabela de símbolos

Entrada (Symbol). Cada identificador distinto corresponde a um único objeto com os atributos: `name` (lexema), `type` (`integer`, `real` ou `None` enquanto não declarado), `lineno` (linha da declaração ou do primeiro uso), `scope` (a `SymbolTable` em que vive), `offset` (deslocamento relativo no escopo) e `declared`. O offset é acumulado pela largura do tipo (`integer = 4`, `real = 8`).

Ponteiros. O valor do token `ID` é o objeto `Symbol`. Duas ocorrências do mesmo nome (inclusive com caixa diferente, pois a chave é `casefold()`) resultam no mesmo ponteiro.

Ambientes encadeados. `SymbolTable` é uma tabela hash com referência `parent` para o ambiente envolvente e campo `level`. O `lookup` sobe de filho para pai até o escopo global. `ScopeManager` mantém a pilha: `enter_scope` cria um filho e `exit_scope` volta ao pai (sem nunca passar do global).

Declarar e usar. O lexer instala o lexema ao vê-lo (`lookup_or_install`), mas a entrada nasce com `declared = False`. Somente a regra de declaração (`declare`) a marca

como declarada. Assim: (a) usar um identificador cuja entrada continua não declarada é o erro variável não declarada; (b) declarar um nome que já está declarado no mesmo escopo é o erro redeclaração, informando a linha da primeira declaração; (c) declarar em um escopo interno um nome que só existe no externo é permitido e cria uma entrada local (sombreamento).

Blocos e ambientes. Cada `begin ... end` abre um ambiente filho: a regra vazia `block_enter` (regra 48), reduzida logo após o `BEGIN`, chama `enter_scope`, e a redução de `block` chama `exit_scope`. O lexer continua sob demanda, e os nomes declarados no ambiente externo permanecem visíveis nos blocos internos pela cadeia `parent`. Na gramática básica, as declarações globais aparecem antes do bloco principal. Na extensão com funções e procedimentos, cada subprograma possui seu próprio ambiente e suas declarações locais são associadas a esse escopo. Os blocos de comandos também podem criar ambientes filhos, mantendo a busca de nomes pela cadeia de ambientes.

5 Gramática e parser LALR(1)

A gramática do enunciado foi usada como base, mas o parser do projeto também reconhece extensões. A versão efetivamente carregada pelo PLY contém 124 produções numeradas de 0 a 123 em `parser.out` e gera 247 estados, numerados de 0 a 246. Portanto, a gramática efetiva é maior que a gramática mínima de Pascal-S apresentada na especificação.

Na parte básica, as listas são construídas com recursão à esquerda, por exemplo `stmt_list : stmt_list SEMICOLON stmt`. A hierarquia `expr`, `simple_expr`, `term` e `factor` expressa os níveis de precedência dos operadores aritméticos e relacionais. As regras de recuperação usam o símbolo especial `error` do PLY.

Extensões sintáticas incluídas. Além de atribuições, condicionais, laços e `writeln`, a gramática aceita `for`/`to`/`downto`, `repeat`/`until`, os operadores inteiros `div` e `mod`, declarações `const`, a cláusula opcional `uses`, declarações `type`, tipos nomeados, vetores multidimensionais, registros, strings e ponteiros; declarações e chamadas de funções e procedimentos, inclusive parâmetros por referência `var`; chamadas de subprogramas como comandos e expressões; acesso a campos, indexação e desreferência de ponteiros; e comandos `new` e `dispose`. Essas construções são extensões do projeto, não produções exigidas explicitamente pela gramática mínima do enunciado.

Precedência e associatividade. Os operadores são organizados pela estrutura das produções. Para o caso do *else pendente*, o parser declara `IFX` com precedência inferior a `ELSE`; a regra do `if` sem `else` recebe `%prec IFX`. Assim, diante de `ELSE`, o parser desloca o token e associa o `else` ao `if` mais próximo ainda sem alternativa.

6 Verificação semântica de tipos

As ações das regras sintáticas calculam o tipo sintetizado de cada expressão (avaliação dirigida por sintaxe, atributo sintetizado em pilha). Os tipos incluem `integer`, `real` e `boolean`; diferentemente do mínimo descrito no enunciado, a implementação permite declarar variáveis booleanas e usar as constantes `true` e `false`. Os tipos adicionais `string`, vetores, registros e ponteiros também são verificados pelas regras específicas de

suas construções. O valor `None` marca uma subexpressão que já gerou erro e é propagado em silêncio, evitando mensagens em cascata.

Construção	Regra aplicada	Erro quando
`+ - * /`	Ambos os operandos do mesmo tipo numérico; resultado = esse tipo.	Tipos distintos (`integer` x `real`) ou operando boolean.
`div / mod`	Ambos os operandos devem ser `integer`; resultado `integer`.	Algum operando não é inteiro.
`= <> < <= > >=`	Ambos do mesmo tipo numérico; resultado boolean.	Tipos distintos ou operando boolean.
`and or`	Ambos boolean; resultado boolean.	Algum operando não é boolean.
`not`	Operando boolean; resultado boolean.	Operando não é boolean.
`x := e`	Tipo de `x` igual ao tipo de `e`.	Tipos diferentes (inclui real em integer e boolean em integer).
`for / repeat`	Controle e limites de `for` são inteiros; condição de `repeat` é booleana.	Tipo incompatível.
`if / while`	Condição boolean.	Condição de outro tipo.
Uso de ID	Entrada deve estar declarada.	Variável não declarada.
Declaração	Nome novo no escopo atual.	Redeclaração no mesmo escopo.

Para as extensões, a análise também verifica que índices de vetores sejam inteiros, que apenas vetores possam ser indexados, que os índices constantes respeitem os limites declarados, que campos acessados existam em registros, que campos não sejam declarados repetidamente e que argumentos de funções/procedimentos correspondam em quantidade e tipo aos parâmetros. Chamadas de procedimentos não são valores de expressão; funções devem respeitar o tipo de retorno. Ponteiros são compatíveis com o mesmo tipo-alvo ou com `nil`; a alocação e a liberação exigem uma variável ponteiro. A cláusula `uses` registra os nomes das unidades e acusa repetições, mas não carrega bibliotecas reais.

Não existe nenhuma conversão implícita. O operador `/` mantém o tipo dos operandos (a divisão de dois inteiros é inteira), o que é coerente com a regra de que o resultado não muda de tipo.

Erros fatais e não fatais. Todos os erros de *tipo* da tabela (operandos incompatíveis, atribuição incompatível, `not`/`and`/`or` com operando não booleano e condição de `if`/`while` não booleana) são fatais, como pede o enunciado ("aborto imediato"): a ação semântica levanta a exceção `CompilationAbort`, que atravessa o PLY e é capturada em `run_file`, que imprime `Compilação abortada no primeiro erro de tipo`, despeja a tabela de símbolos e encerra com FALHA e código de saída 1. Já os erros de *declaração* (variável não declarada, em expressão ou em atribuição, e redeclaração) são registrados e a análise continua, de modo que vários deles podem ser listados em uma mesma execução. Como a tokenização é sob demanda, um erro fatal também impede que o restante do arquivo seja lido.

7 Recuperação de erros em modo pânico

O callback `p_error(p)` reporta a linha, a coluna e o token inesperado e em seguida entra em modo pânico: descarta tokens, obtidos sob demanda do mesmo fluxo preguiçoso, até encontrar um token de sincronização, que é devolvido ao fluxo (pushback) sem ser consumido. Ao retornar, o PLY insere o símbolo especial `error` como lookahead e desempilha estados até encontrar um que aceite `error`, usando as produções de recuperação previstas na gramática. A regra escolhida reduz, a análise segue a partir do sincronizador e novos erros podem ser reportados na mesma execução. O número de tokens descartados é informado no log (`[modo pânico] N token(s) descartado(s)`).

Desvio em relação ao enunciado. O enunciado adota apenas o ponto e vírgula como sincronizador. Aqui o conjunto é `{;, end, until}`. O motivo é gramatical: em Pascal o `;` separa comandos e o último comando de um bloco não o possui. Sem o `end` e o `until` no conjunto, um erro no último comando de um bloco ou de um `repeat` poderia engolir o delimitador e cascatear erros falsos (ou atravessar o programa até o fim do arquivo). O teste `sync_end.pas` (`a :=` seguido de `end`) mostra exatamente esse caso: um único erro, no `end`, e a análise retomada no sincronizador. Quando o próprio token ofensor já é um sincronizador (`;`, `end` ou `until`), nada é descartado.

Garantia de progresso. Se, após a recuperação, o parser voltar a falhar exatamente no mesmo token (por exemplo um `else` solto, que não é sincronizador), esse token é descartado em silêncio, para evitar laço infinito. Esse caso é exercitado por `erro_else_solto`. No fim do arquivo (`p` nulo) o erro é reportado como `fim inesperado do arquivo` e não há recuperação possível.

Erros léxicos e erros sintáticos. Um caractere ilegal é descartado pelo lexer e não chega ao parser. Em `cont := 10 @ 5;` (cenário 2) o parser enxerga `10 5` e reporta também um erro sintático no `5`; os dois erros são reais e coexistem no log. Em seguida o parser se recupera no `;` e o `writeln` seguinte é analisado normalmente.

8 Análise de conflitos shift/reduce

Resultado global. A gramática foi gerada pelo PLY (LALR(1)) com debug habilitado. O único conflito registrado em `parser.out` é o do *else pendente*, resolvido por precedência (linha marcada com `!` no estado 209); não há outros conflitos registrados no arquivo gerado.

O conflito. As regras 84 (`if_stmt -> IF expr THEN stmt`) e 85 (`if_stmt -> IF expr THEN stmt ELSE stmt`) compartilham um prefixo. Depois de analisar o `stmt` do ramo `then`, o parser está em um estado que contém os dois itens LR(0):

```
state 209
    (84) if_stmt -> IF expr THEN stmt .
    (85) if_stmt -> IF expr THEN stmt . ELSE stmt

    END reduce using rule 84 (if_stmt -> IF expr THEN stmt .)
    SEMICOLON reduce using rule 84 (if_stmt -> IF expr THEN stmt .)
    ELSE shift and go to state 233

  ! ELSE [ reduce using rule 84 (if_stmt -> IF expr THEN stmt .) ]
```

Com lookahead `ELSE` há duas ações possíveis: reduzir pela regra 84 (fechar o `if` sem `else`) ou empilhar o `else` (regra 85, estado 233). Isso ocorre porque `ELSE` pertence ao conjunto FOLLOW de `if_stmt` (via `stmt`), já que um `if` aninhado no ramo `then` de outro pode ser seguido pelo `else` do externo. A gramática é, portanto, ambígua nesse ponto: `if a then if b then x else y` admite duas árvores.

Resolução. Foi declarada a precedência `IFX < ELSE` (ambos não associativos) e a regra 84 recebeu `%prec IFX`. Como o token `ELSE` tem precedência maior que a regra, o PLY resolve o conflito a favor do shift: cada `else` casa com o `if` mais próximo ainda sem `else` (a interpretação padrão de Pascal). A linha com `!` no estado 209 é a ação descartada. O teste `sucesso_else_pendente.pas` exercita exatamente essa construção.

Por que os demais casos não conflitam. (i) A produção vazia `stmt -> vazio` convive com o `if` sem conflito: no estado correspondente a `IF expr THEN . stmt` ela é reduzida apenas diante de `ELSE`, `END` e `;`, tokens que nenhum outro item do estado empilha. O comportamento da produção vazia depende do lookahead e é descrito pelas tabelas de estados geradas pelo PLY. A verificação de `parser.out` não mostra conflito adicional associado à produção vazia. (ii) Expressões: como a hierarquia `expr`, `simple_expr`, `term` e `factor` já separa os níveis, operadores de mesma classe usam recursão à esquerda (associatividade à esquerda) e operadores de classes diferentes não disputam o mesmo item. (iii) As produções com `error` usam o sincronizador seguinte (`decl_group -> error ';'`) ou aparecem como alternativa de `stmt` e dependem de `stmt_list` para o delimitador, então não criam ambiguidades novas. (iv) A regra vazia `block_enter` é a única ação possível logo após o `BEGIN`, portanto sua redução não compete com nenhum deslocamento.

Alternativa descartada. Reescrever a gramática com `matched_stmt` / `unmatched_stmt` eliminaria o conflito sem precedências, mas aumentaria o número de regras e se afastaria da gramática do enunciado; a precedência é a solução padrão em geradores LALR e deixa o parser mais próximo da especificação.

9 Casos de teste e resultados

O pacote contém 78 arquivos de entrada em `testes/` e 78 logs em `logs/`. O script `python run_tests.py` executa a bateria e regrava os logs. A tabela abaixo apresenta uma seleção representativa dos cenários; os resultados são os registrados nos logs incluídos no pacote. Os três primeiros cenários correspondem aos casos obrigatórios do enunciado.

Teste	O que exercita	Resultado registrado
`sucesso_semantico.pas`	Cenário 1: tipos homogêneos, while, writeln.	SUCESSO (código 0).
`erro_lexico.pas`	Cenário 2: caractere ilegal '@' (5:14).	1 léxico + 1 sintático (5:16, efeito do descarte); FALHA.
`erro_tipo.pas`	Cenário 3: real atribuído a integer.	1 semântico (linha 8), compilação abortada; FALHA.
`erro_sintatico.pas`	Expressão ausente (`valor := ;`).	1 sintático (5:12), análise retomada; FALHA.

`erro_redeclaracao.pas`	Mesmo nome declarado duas vezes.	1 semântico (linha 4; 1ª declaração na linha 3); FALHA.
`erro_nao_declarada.pas`	Uso sem declaração.	1 semântico (linha 5), sem abortar; FALHA.
`erros_multiplos.pas`	Dois erros sintáticos e depois um erro de tipo.	Sintáticos em 6:8 e 7:12, semântico na linha 8 (aborta; a linha 9 não é lida); FALHA.
`recuperacao_semantica.pas`	Erro sintático seguido de erro de tipo.	1 sintático (5:16) + 1 semântico (linha 6); FALHA.
`erro_operador_duplo.pas`	Operador binário seguido de operador.	Sintáticos em 6:16 e 7:16, semântico na linha 8 (aborta); FALHA.
`erro_duas_na_mesma_linha.pas`	Dois erros sintáticos na mesma linha.	Sintáticos em 4:12 e 4:26, ambos recuperados; FALHA.
`erro_else_solto`	`else` sem `if`.	1 sintático (6:5), 3 tokens descartados; FALHA.
`sync_end.pas`	Erro no último comando do bloco.	1 sintático (6:1, no `end`), análise retomada; FALHA.
`erro_cabecalho.pas`	`program 123;`.	1 sintático (1:9); FALHA.
`erro_declaracao.pas`	`var x integer;` (dois-pontos ausente).	1 sintático (2:7); `y` é declarada normalmente; FALHA.
`erro_sem_end.pas`	Begin sem end.	1 sintático, fim inesperado do arquivo (6:1); FALHA.
`erro_sem_ponto.pas`	Ponto final ausente.	1 sintático, fim inesperado do arquivo (6:1); FALHA.
`erro_logicos_tipos.pas`	and/or/not com operandos integer.	1 semântico (linha 5, `and`), aborta; FALHA.
`erro_condicao_inteira.pas`	while/if com condição integer.	1 semântico (linha 5, `while`), aborta; FALHA.
`lex_extra.pas`	Palavras reservadas em caixas variadas, identificadores com _ e dígito, comentários, `$ # !` e comentário não fechado.	O erro de tipo da linha 6 (`x_1 := 3.14`) aborta antes que o lexer chegue aos caracteres ilegais; 1 semântico; FALHA. Os erros léxicos desse arquivo são listados por `python lex.py testes/lex_extra.pas`.
`sucesso_else_pendente.pas`	If aninhado com um else (else pendente).	SUCESSO.
`sucesso_blocos_aninhados.pas`	Begin/end aninhados, reservadas em caixas variadas, comentário //.	SUCESSO.
`sucesso_stmt_vazio.pas`	Comandos vazios (`;;` e `;` antes do `end`).	SUCESSO.

Exemplo - cenário 3 (log completo, `erro_tipo`):

```
Erro semântico (linha 8): atribuição incompatível: 'cont' éinteger, mas a
    expressão éreal.
Compilação abortada no primeiro erro de tipo.

TABELA DE SIMBOLOS
[escopo 'global' nivel 0]
  nome tipo offset linha
  cont integer 0 3
  acumulador real 4 4
  taxa real 12 4

Erros léxicos: 0
Erros sintáticos: 0
Erros semânticos: 1

Resultado: FALHA
```

Exemplo - vários erros em uma execução (`erros_multiplos`):

```
Erro sintático [linha 6, coluna 8]: token inesperado SEMICOLON (';').
Recuperação sintática: análise retomada no sincronizador.
Erro sintático [linha 7, coluna 12]: token inesperado TIMES ('*').
    [modo pânico] 1 token(s) descartado(s).
Recuperação sintática: análise retomada no sincronizador.
Erro semântico (linha 8): atribuição incompatível: 'x' éreal, mas a expressão é
    integer.
Compilação abortada no primeiro erro de tipo.
```

10 Limitações e conclusão

Limitações conhecidas. (i) A cláusula `uses` registra os nomes das unidades, mas não implementa nem carrega as bibliotecas correspondentes. (ii) Os parâmetros de funções e procedimentos podem ser passados por valor ou por referência com `var`; subprogramas aninhados não fazem parte da implementação. (iii) A linguagem implementada é um subconjunto ampliado de Pascal, não um compilador Pascal completo; construções não implementadas, como `write` e `readln`, não devem ser consideradas suportadas. Já `for`, `repeat`, `div`, `mod`, `const` e parâmetros `var` estão implementados e possuem testes específicos no pacote. (iv) Erros semânticos de tipo são fatais, então os erros posteriores no arquivo não são necessariamente reportados. (v) Um caractere ilegal é descartado pelo lexer e pode provocar também um erro sintático consequente. (vi) O arquivo é carregado em uma string, conforme a interface do PLY, mas a tokenização ocorre sob demanda.

Conclusão. A implementação contempla os principais requisitos do TP1: tokenização sob demanda com informação de linha e coluna, tabela de símbolos com referências a objetos e ambientes encadeados, parser LALR(1), verificação sem conversões implícitas e recuperação sintática em modo pânico. O conflito do *else pendente* foi identificado no estado 209 do `parser.out` e resolvido por precedência. O projeto também implementa extensões como vetores, matrizes, registros, ponteiros, strings, tipos definidos pelo usuário,

funções e procedimentos, além de `for`, `repeat`, `div`, `mod`, `const` e parâmetros `var`. O pacote contém 78 testes e seus logs correspondentes; os resultados registrados devem ser interpretados conforme cada caso, distinguindo programas válidos dos testes que verificam a detecção de erros.

A Execução

```
python3 -m pip install ply==3.11
python3 yacc.py testes/sucesso_semantico.pas
python3 lex.py testes/lex_extra.pas
python3 run_tests.py
```